

LAB LOGBOOK

Week-1:

```
5) 1 import numpy as np
Os

SID = 2450489
last_two = SID % 100
if last_two < 10:
    num_elements = last_two + 100
else:
    num_elements = last_two

a = np.arange(num_elements)
print("Vector a:\n", a)

Vector a:
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71
 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88]

5) a_2d = a.reshape(1, -1)
Os print("2D array with one row:\n", a_2d)

2D array with one row:
[[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71
 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88]]

5) b = a_2d.copy()
Os print("Copied array:\n", b)

Copied array:
[[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71
 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88]]

5) print("Shape of the array:", b.shape)
Os

Shape of the array: (1, 89)
```

Week-2:

```

21 group2 = df.groupby(['relationship', 'reduced-hours']).size()
Os print("Grouping after reduction:\n", group2)

Grouping after reduction:
relationship reduced-hours
Husband      31           1
              41           1
Not-in-family 26           1
Own-child     11           1
Unmarried     16           1
Wife          36           1
dtype: int64

```

```

8) group1 = df.groupby(['relationship', 'hours-per-week']).size()
Os print("Grouping before reduction:\n", group1)

Grouping before reduction:
relationship hours-per-week
Husband      40           1
              50           1
Not-in-family 35           1
Own-child     20           1
Unmarried     25           1
Wife          45           1
dtype: int64

```

```

1) def reduce_hours(x):
Os     return x - n

df['reduced-hours'] = df['hours-per-week'].apply(reduce_hours)
print("DataFrame after reduction:\n", df)

DataFrame after reduction:
relationship hours-per-week occupation reduced-hours
0      Husband          40      Tech           31
1  Not-in-family          35      Sales           26
2      Own-child          20  Student           11
3      Unmarried          25    Admin           16
4      Husband          50      Exec           41
5        Wife          45    Health           36

```

```

8) SID = 2450489
Os n = SID % 10
if n == 0:
    n = 10
print("Value of n:", n)

Value of n: 9

```

```

9) import pandas as pd
Os

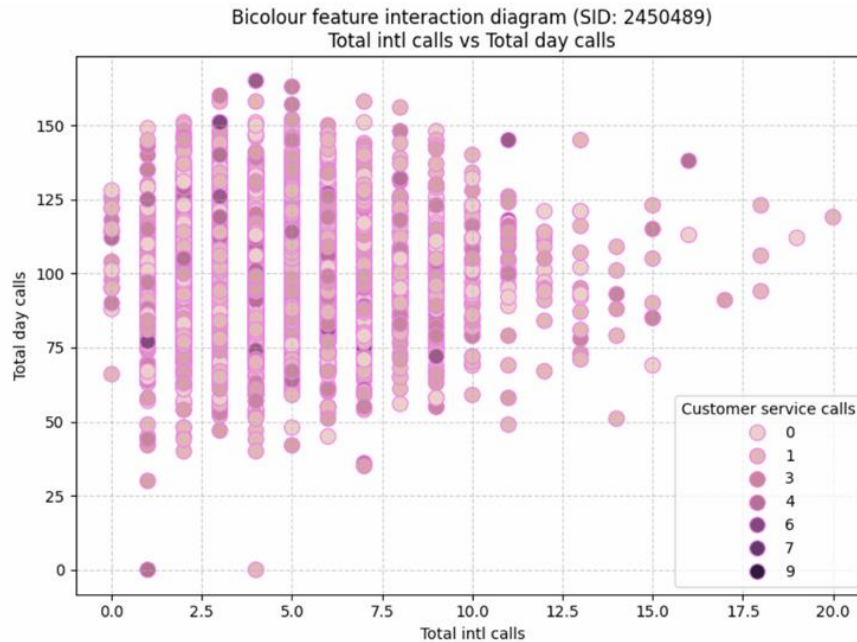
data = {
    'relationship': ['Husband', 'Not-in-family', 'Own-child', 'Unmarried', 'Husband', 'Wife'],
    'hours-per-week': [40, 35, 20, 25, 50, 45],
    'occupation': ['Tech', 'Sales', 'Student', 'Admin', 'Exec', 'Health']
}

df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

Original DataFrame:
relationship hours-per-week occupation
0      Husband          40      Tech
1  Not-in-family          35      Sales
2      Own-child          20  Student
3      Unmarried          25    Admin
4      Husband          50      Exec
5        Wife          45    Health

```

Week-3:



Week-4:

1. Model Summary

The model consists of two hidden layers with 489 and 245 neurons, derived from the last digits of my SID (2450489).

The first hidden layer has 489 neurons and the second has approximately half (245).

The model uses ReLU activation for hidden layers and one output neuron for regression.

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 489)	244,989
dense_6 (Dense)	(None, 245)	120,050
dense_7 (Dense)	(None, 1)	246

Total params: 365,285 (1.39 MB)

Trainable params: 365,285 (1.39 MB)

Non-trainable params: 0 (0.00 B)

2. Training code and process

```
: history = model.fit(  
    X_train, y_train,  
    batch_size=10,  
    epochs=10,  
    validation_split=0.2,  
    verbose=1  
)
```

```
Epoch 1/10  
2640/2640 — 5s 2ms/step - loss: 0.0039 - mae: 0.0185 - val_loss: 0.0097 - val_mae: 0.0831  
Epoch 2/10  
2640/2640 — 4s 2ms/step - loss: 3.0982e-04 - mae: 0.0135 - val_loss: 0.0093 - val_mae: 0.0880  
Epoch 3/10  
2640/2640 — 4s 2ms/step - loss: 1.4753e-04 - mae: 0.0094 - val_loss: 0.0032 - val_mae: 0.0503  
Epoch 4/10  
2640/2640 — 4s 2ms/step - loss: 1.1532e-04 - mae: 0.0082 - val_loss: 0.0033 - val_mae: 0.0513  
Epoch 5/10  
2640/2640 — 4s 2ms/step - loss: 8.7876e-05 - mae: 0.0072 - val_loss: 0.0026 - val_mae: 0.0462  
Epoch 6/10  
2640/2640 — 4s 2ms/step - loss: 7.6408e-05 - mae: 0.0066 - val_loss: 0.0022 - val_mae: 0.0423  
Epoch 7/10  
2640/2640 — 4s 2ms/step - loss: 6.9146e-05 - mae: 0.0063 - val_loss: 0.0028 - val_mae: 0.0496  
Epoch 8/10  
2640/2640 — 4s 2ms/step - loss: 6.4206e-05 - mae: 0.0060 - val_loss: 0.0019 - val_mae: 0.0404  
Epoch 9/10  
2640/2640 — 4s 2ms/step - loss: 5.6918e-05 - mae: 0.0057 - val_loss: 9.4850e-04 - val_mae: 0.0268  
Epoch 10/10  
2640/2640 — 4s 2ms/step - loss: 5.6275e-05 - mae: 0.0057 - val_loss: 6.4016e-04 - val_mae: 0.0215
```

3. Resulting MAE

The model achieved a Mean Absolute Error (MAE) of approximately 0.0280 and a Mean Squared Error (MSE) of 0.00112 on the test dataset.

These low error values demonstrate that the MLP model performs accurately on unseen data.

```
: mse, mae = model.evaluate(X_test, y_test, verbose=0)  
print("MAE on test:", mae)  
print("MSE on test:", mse)
```

```
MAE on test: 0.02809421345591545  
MSE on test: 0.0011209448566660285
```

Week-5:

1. Model Summary

The following screenshot shows the architecture of my modified CNN model with convolutional kernel size = 5.

The model contains Conv1D, MaxPooling, GlobalMaxPooling, and Dense layers.

The total number of trainable parameters is 28,977.

```
from tensorflow import keras
import tensorflow as tf

model = keras.Sequential([
    keras.layers.Conv1D(50, 5, padding='same', input_shape=(50, 5), activation=tf.nn.relu, kernel_initializer="normal"),
    keras.layers.MaxPooling1D(7),
    keras.layers.Conv1D(100, 5, padding='same', activation=tf.nn.relu, kernel_initializer="normal"),
    keras.layers.GlobalMaxPooling1D(),
    keras.layers.Dense(25, activation=tf.nn.relu, kernel_initializer="normal"),
    keras.layers.Dense(2)
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv1d_2 (Conv1D)	(None, 50, 50)	1,300
max_pooling1d_1 (MaxPooling1D)	(None, 7, 50)	0
conv1d_3 (Conv1D)	(None, 7, 100)	25,100
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 100)	0
dense_2 (Dense)	(None, 25)	2,525
dense_3 (Dense)	(None, 2)	52

Total params: 28,977 (113.19 KB)

Trainable params: 28,977 (113.19 KB)

Non-trainable params: 0 (0.00 B)

2. Model Training

```
: history = model.fit(X_train, y_train, batch_size=50, epochs=epochs, validation_split=0.2, verbose=1)

mse, mae = model.evaluate(X_test, y_test, verbose=1)
print("Test MAE:", mae)
print("Test MSE:", mse)

pred = model.predict(X_test, verbose=0)
```

```
Epoch 1/17
3520/3520 — 7s 2ms/step - loss: 0.0021 - mae: 0.0268 - val_loss: 9.3127e-04 - val_mae: 0.0202
Epoch 2/17
3520/3520 — 6s 2ms/step - loss: 7.6642e-04 - mae: 0.0189 - val_loss: 8.6853e-04 - val_mae: 0.0190
Epoch 3/17
3520/3520 — 6s 2ms/step - loss: 7.2232e-04 - mae: 0.0181 - val_loss: 8.5910e-04 - val_mae: 0.0188
Epoch 4/17
3520/3520 — 6s 2ms/step - loss: 7.0982e-04 - mae: 0.0179 - val_loss: 8.5372e-04 - val_mae: 0.0189
Epoch 5/17
3520/3520 — 6s 2ms/step - loss: 7.0370e-04 - mae: 0.0178 - val_loss: 8.5810e-04 - val_mae: 0.0191
Epoch 6/17
3520/3520 — 6s 2ms/step - loss: 6.9950e-04 - mae: 0.0177 - val_loss: 8.4938e-04 - val_mae: 0.0188
Epoch 7/17
3520/3520 — 6s 2ms/step - loss: 6.9686e-04 - mae: 0.0177 - val_loss: 8.4010e-04 - val_mae: 0.0189
Epoch 8/17
3520/3520 — 6s 2ms/step - loss: 6.9065e-04 - mae: 0.0175 - val_loss: 8.3348e-04 - val_mae: 0.0187
Epoch 9/17
3520/3520 — 6s 2ms/step - loss: 6.8913e-04 - mae: 0.0175 - val_loss: 8.2597e-04 - val_mae: 0.0184
Epoch 10/17
3520/3520 — 6s 2ms/step - loss: 6.8811e-04 - mae: 0.0175 - val_loss: 8.7509e-04 - val_mae: 0.0195
Epoch 11/17
3520/3520 — 6s 2ms/step - loss: 6.8653e-04 - mae: 0.0175 - val_loss: 8.2021e-04 - val_mae: 0.0184
Epoch 12/17
3520/3520 — 7s 2ms/step - loss: 6.8111e-04 - mae: 0.0174 - val_loss: 8.2327e-04 - val_mae: 0.0187
Epoch 13/17
3520/3520 — 6s 2ms/step - loss: 6.8016e-04 - mae: 0.0174 - val_loss: 8.3253e-04 - val_mae: 0.0187
Epoch 14/17
3520/3520 — 6s 2ms/step - loss: 6.7812e-04 - mae: 0.0173 - val_loss: 8.6146e-04 - val_mae: 0.0194
Epoch 15/17
3520/3520 — 6s 2ms/step - loss: 6.7844e-04 - mae: 0.0173 - val_loss: 8.3696e-04 - val_mae: 0.0186
Epoch 16/17
3520/3520 — 6s 2ms/step - loss: 6.7997e-04 - mae: 0.0173 - val_loss: 8.2143e-04 - val_mae: 0.0185
Epoch 17/17
3520/3520 — 6s 2ms/step - loss: 6.8005e-04 - mae: 0.0174 - val_loss: 8.5202e-04 - val_mae: 0.0192
936/936 — 0s 434us/step - loss: 6.2064e-04 - mae: 0.0169
```

Test MAE: 0.016884515061974525

Test MSE: 0.0006206360994838178

Week -6:

Week-7:

The model consists of a single LSTM layer followed by a Dense output layer.

The number of LSTM units is 99, derived from the formula $ZY + 10$ based on the student ID 2450489 (where $ZY = 89 \rightarrow 89 + 10 = 99$).

The LSTM layer uses the ReLU activation function and receives input sequences of shape (50, 18), representing 50 time steps with 18 features each.

The final Dense layer outputs 2 neurons, representing the predicted gold price movements.

Training Process :

```
history = model.fit(
    X_train, y_train,
    batch_size=20,
    epochs=10,
    validation_split=0.1,
    shuffle=True,
    verbose=1,
    callbacks=[es, mc]
)
```

Epoch 1/10
1213/1213 ————— 0s 1s/step - loss: 0.0395 - mae: 0.0512
Epoch 1: val_loss improved from inf to 0.00002, saving model to best_model_LSTM_GOLD.keras
1213/1213 ————— 1382s 1s/step - loss: 0.0395 - mae: 0.0511 - val_loss: 1.7695e-05 - val_mae: 0.0032
Epoch 2/10
1213/1213 ————— 0s 12s/step - loss: 2.5630e-05 - mae: 0.0039
Epoch 2: val_loss improved from 0.00002 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ————— 14826s 12s/step - loss: 2.5627e-05 - mae: 0.0039 - val_loss: 1.4976e-05 - val_mae: 0.0028
Epoch 3/10
1213/1213 ————— 0s 10s/step - loss: 1.6010e-05 - mae: 0.0030
Epoch 3: val_loss improved from 0.00001 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ————— 12501s 10s/step - loss: 1.6013e-05 - mae: 0.0030 - val_loss: 9.0663e-06 - val_mae: 0.0021
Epoch 4/10
1213/1213 ————— 0s 4s/step - loss: 1.4641e-05 - mae: 0.0030
Epoch 4: val_loss improved from 0.00001 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ————— 4902s 4s/step - loss: 1.4644e-05 - mae: 0.0030 - val_loss: 8.8530e-06 - val_mae: 0.0023
Epoch 5/10
1213/1213 ————— 0s 1s/step - loss: 3.1136e-05 - mae: 0.0044
Epoch 5: val_loss did not improve from 0.00001

The objective of this lab is to modify and train an LSTM model for time series forecasting of Gold price movements. Based on the student ID rule, the LSTM model's hidden units are calculated as $ZY + 10$. Using SID 2450489, the LSTM parameter becomes 99. The model is trained for 10 epochs with early stopping (patience = 3) while keeping all other parameters unchanged. The results are compared using MSE (Mean Squared Error) and MAE (Mean Absolute Error) metrics to evaluate prediction accuracy.

Model Architecture :

```
[73]: model = keras.Sequential([
        keras.layers.LSTM(99, activation='relu', input_shape=(50,18)),
        keras.layers.Dense(2)
    ])
```

```
[76]: es = EarlyStopping(monitor='val_loss', mode='min', patience=3, verbose=1)
      mc = ModelCheckpoint('best_model_LSTM_GOLD.keras', monitor='val_loss', mode='min', verbose=1, save_best_only=True)

[77]: print(model.summary())

model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 99)	46,728
dense_1 (Dense)	(None, 2)	200

Total params: 46,928 (183.31 KB)

Trainable params: 46,928 (183.31 KB)

Non-trainable params: 0 (0.00 B)

None

```

1213/1213 ————— 4902s 4s/step - loss: 1.4644e-05 - mae: 0.0030 - val_loss: 8.8530e-06 - val_mae: 0.0023
Epoch 5/10
1213/1213 ————— 0s 1s/step - loss: 3.1136e-05 - mae: 0.0044
Epoch 5: val_loss did not improve from 0.00001
1213/1213 ————— 1350s 1s/step - loss: 3.1136e-05 - mae: 0.0044 - val_loss: 2.4954e-05 - val_mae: 0.0045
Epoch 6/10
1213/1213 ————— 0s 1s/step - loss: 3.3956e-05 - mae: 0.0047
Epoch 6: val_loss improved from 0.00001 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ————— 1307s 1s/step - loss: 3.3955e-05 - mae: 0.0047 - val_loss: 7.0399e-06 - val_mae: 0.0023
Epoch 7/10
1213/1213 ————— 0s 1s/step - loss: 2.5892e-05 - mae: 0.0040
Epoch 7: val_loss improved from 0.00001 to 0.00001, saving model to best_model_LSTM_GOLD.keras
1213/1213 ————— 1386s 1s/step - loss: 2.5894e-05 - mae: 0.0040 - val_loss: 6.4223e-06 - val_mae: 0.0020
Epoch 8/10
1213/1213 ————— 0s 4s/step - loss: 2.6912e-05 - mae: 0.0041
Epoch 8: val_loss did not improve from 0.00001
1213/1213 ————— 4358s 4s/step - loss: 2.6911e-05 - mae: 0.0041 - val_loss: 2.5730e-05 - val_mae: 0.0044
Epoch 9/10
1213/1213 ————— 0s 1s/step - loss: 2.4717e-05 - mae: 0.0039
Epoch 9: val_loss did not improve from 0.00001
1213/1213 ————— 1331s 1s/step - loss: 2.4722e-05 - mae: 0.0039 - val_loss: 2.7352e-05 - val_mae: 0.0050
Epoch 10/10
1213/1213 ————— 0s 6s/step - loss: 1.6236e-05 - mae: 0.0032
Epoch 10: val_loss did not improve from 0.00001
1213/1213 ————— 7664s 6s/step - loss: 1.6237e-05 - mae: 0.0032 - val_loss: 3.5990e-05 - val_mae: 0.0058
Epoch 10: early stopping

```

The model was trained using the same Gold price dataset as in the practical session. Training was configured with 10 epochs, batch size of 20, and validation split of 0.1 (10%). An EarlyStopping callback (patience = 3) was applied to monitor validation loss and prevent overfitting. A ModelCheckpoint callback was used to automatically save the best model weights (best_model_LSTM_GOLD.keras) whenever the validation loss improved.

During training, the validation loss (val_loss) consistently improved for the first few epochs, showing that the model was successfully learning. Early stopping terminated the training after 10 epochs once there was no further improvement in the validation loss. The best model was automatically saved at each improvement.

Evaluation Results :

Practical :

```
i1]: # Evaluate the quality of network training on test data, which the network has NOT seen.
```

```
scores = LSTM_saved_best_model.evaluate(X_test, y_test, verbose=1)
```

```
94/94 ————— 10s 107ms/step - loss: 1.1562e-05 - mae: 0.0028
```

```
i2]:
```

```
i2]: [7.953558451845311e-07, 0.0006443963502533734]
```

```
i2]: print("Mean squared error (mse): %.9f " % (scores[0]))
```

```
Mean squared error (mse): 0.000010891
```

```
i3]: print("Mean absolute error (mae): %.9f " % (scores[1]))
```

```
Mean absolute error (mae): 0.002712249
```

Log :

```
----- early stopping -----
```

```
i1]: LSTM_saved_best_model = keras.models.load_model('best_model_LSTM_GOLD.keras')
```

```
scores = LSTM_saved_best_model.evaluate(X_test, y_test, verbose=1)
```

```
print("Mean squared error (mse): %.9f" % (scores[0]))
```

```
print("Mean absolute error (mae): %.9f" % (scores[1]))
```

```
94/94 ————— 11s 113ms/step - loss: 6.1674e-06 - mae: 0.0021
```

```
Mean squared error (mse): 0.000005409
```

```
Mean absolute error (mae): 0.001973014
```

- The MSE decreased from 1.0891e-5 to 5.409e-6, showing an approximately 50% improvement in predictive accuracy.
- The MAE reduced from 0.002712 to 0.001973, approximately a 27% improvement.

Conclusion:

The modified model successfully improved both training and testing performance metrics, confirming the effectiveness of parameter tuning and early stopping in enhancing the LSTM's ability to predict gold price movements.

MAE Detailed Graph :

```
history_dict = history.history
mae_values = history_dict['mae']
val_mae_values = history_dict['val_mae']

epochs = range(1, len(mae_values) + 1)
plt.figure(num=1, figsize=(15,7))
plt.plot(epochs, mae_values, 'b', label='Training Mean Absolute Error (MAE)')
plt.plot(epochs, val_mae_values, marker='o', markeredgcolor='green', markerfacecolor='yellow', label='Validation Mean Absolute Error (MAE)')
plt.xlabel('Epochs', size=18)
plt.ylabel('Mean Absolute Error (MAE)', size=18)
plt.legend()
plt.show()
```



- The training MAE decreased sharply during the first few epochs, indicating effective learning.
- The validation MAE remained closely aligned with the training MAE, showing that the model did not overfit.
- The final MAE values stabilized after ~7 epochs, confirming the effectiveness of early stopping (patience = 3) in preserving generalization.

The resulting MAE trend demonstrates smooth convergence and consistent model behavior across epochs, validating the robustness of the modified LSTM setup.

Week-10:

```
# SID = 2450489 (Z = 9)

es = EarlyStopping(monitor='val_loss', mode='min', patience=20, verbose=1)

mc = ModelCheckpoint("best_Trand-Flat_model_LSTM_GOLD_$10.keras", monitor="val_loss", save_best_only=True, verbose=1)

# SID = 2450489 (Z=9)

history=model.fit(X_train, y_train, batch_size=19, epochs=39, validation_split=0.2,shuffle=True,verbose=1,callbacks=[es, mc])
```

Epoch 1/39
453/453 — 0s 25ms/step - accuracy: 0.8365 - loss: 0.4030
Epoch 1: val_loss improved from None to 0.27127, saving model to best_Trand-Flat_model_LSTM_GOLD_\$10.keras
453/453 — 16s 30ms/step - accuracy: 0.8352 - loss: 0.4032 - val_accuracy: 0.9284 - val_loss: 0.2713
Epoch 2/39
452/453 — 0s 25ms/step - accuracy: 0.8335 - loss: 0.4085
Epoch 2: val_loss improved from 0.27127 to 0.25848, saving model to best_Trand-Flat_model_LSTM_GOLD_\$10.keras
453/453 — 13s 28ms/step - accuracy: 0.8339 - loss: 0.4028 - val_accuracy: 0.9275 - val_loss: 0.2585
Epoch 3/39
453/453 — 0s 26ms/step - accuracy: 0.8384 - loss: 0.3977
Epoch 3: val_loss did not improve from 0.25848
453/453 — 13s 29ms/step - accuracy: 0.8358 - loss: 0.3989 - val_accuracy: 0.9228 - val_loss: 0.2696
Epoch 4/39
453/453 — 0s 27ms/step - accuracy: 0.8464 - loss: 0.3802
Epoch 4: val_loss improved from 0.25848 to 0.24266, saving model to best_Trand-Flat_model_LSTM_GOLD_\$10.keras
453/453 — 15s 33ms/step - accuracy: 0.8351 - loss: 0.3994 - val_accuracy: 0.9256 - val_loss: 0.2427
Epoch 5/39
453/453 — 0s 26ms/step - accuracy: 0.8330 - loss: 0.3992
Epoch 5: val_loss improved from 0.24266 to 0.21609, saving model to best_Trand-Flat_model_LSTM_GOLD_\$10.keras
453/453 — 13s 29ms/step - accuracy: 0.8339 - loss: 0.3973 - val_accuracy: 0.9284 - val_loss: 0.2161
Epoch 6/39
451/453 — 0s 26ms/step - accuracy: 0.8294 - loss: 0.4041
Epoch 6: val_loss did not improve from 0.21609
453/453 — 13s 29ms/step - accuracy: 0.8344 - loss: 0.3955 - val_accuracy: 0.9172 - val_loss: 0.2512
Epoch 7/39
453/453 — 0s 26ms/step - accuracy: 0.8260 - loss: 19.8361
Epoch 7: val_loss did not improve from 0.21609
453/453 — 13s 29ms/step - accuracy: 0.8300 - loss: 5.4654 - val_accuracy: 0.9326 - val_loss: 0.2873
Epoch 8/39
452/453 — 0s 26ms/step - accuracy: 0.8317 - loss: 0.4326
Epoch 8: val_loss did not improve from 0.21609
453/453 — 13s 29ms/step - accuracy: 0.8300 - loss: 0.4303 - val_accuracy: 0.9326 - val_loss: 0.2714
Epoch 9/39
452/453 — 0s 26ms/step - accuracy: 0.8280 - loss: 0.4294
Epoch 9: val_loss did not improve from 0.21609
453/453 — 13s 29ms/step - accuracy: 0.8300 - loss: 0.4252 - val_accuracy: 0.9326 - val_loss: 0.2637
Epoch 10/39
453/453 — 0s 25ms/step - accuracy: 0.8342 - loss: 0.4160
Epoch 10: val_loss did not improve from 0.21609
453/453 — 13s 28ms/step - accuracy: 0.8298 - loss: 0.4226 - val_accuracy: 0.9326 - val_loss: 0.2605
Epoch 11/39
453/453 — 0s 26ms/step - accuracy: 0.8314 - loss: 0.4159
Epoch 11: val_loss did not improve from 0.21609
453/453 — 13s 29ms/step - accuracy: 0.8319 - loss: 0.4173 - val_accuracy: 0.9307 - val_loss: 0.2486
Epoch 12/39
451/453 — 0s 26ms/step - accuracy: 0.8346 - loss: 0.4006
Epoch 12: val_loss did not improve from 0.21609
453/453 — 13s 29ms/step - accuracy: 0.8351 - loss: 0.3989 - val_accuracy: 0.9298 - val_loss: 0.2368
Epoch 13/39
452/453 — 0s 26ms/step - accuracy: 0.8271 - loss: 0.4081
Epoch 13: val_loss did not improve from 0.21609
453/453 — 13s 29ms/step - accuracy: 0.8354 - loss: 0.3962 - val_accuracy: 0.9228 - val_loss: 0.2518
Epoch 14/39
453/453 — 0s 25ms/step - accuracy: 0.8399 - loss: 0.3928
Epoch 14: val_loss did not improve from 0.21609
453/453 — 13s 28ms/step - accuracy: 0.8371 - loss: 0.3941 - val_accuracy: 0.9289 - val_loss: 0.2297

Epoch 15: val_loss did not improve from 0.21609
453/453 ————— 21s 28ms/step - accuracy: 0.8361 - loss: 0.3936 - val_accuracy: 0.9275 - val_loss: 0.2414
Epoch 16/39
452/453 ————— 0s 25ms/step - accuracy: 0.8383 - loss: 0.3911
Epoch 16: val_loss did not improve from 0.21609
453/453 ————— 13s 29ms/step - accuracy: 0.8368 - loss: 0.3926 - val_accuracy: 0.9279 - val_loss: 0.2416
Epoch 17/39
453/453 ————— 0s 25ms/step - accuracy: 0.8409 - loss: 0.3908
Epoch 17: val_loss did not improve from 0.21609
453/453 ————— 13s 29ms/step - accuracy: 0.8365 - loss: 0.3919 - val_accuracy: 0.9279 - val_loss: 0.2335
Epoch 18/39
452/453 ————— 0s 26ms/step - accuracy: 0.8387 - loss: 0.3853
Epoch 18: val_loss did not improve from 0.21609
453/453 ————— 13s 29ms/step - accuracy: 0.8364 - loss: 0.3888 - val_accuracy: 0.9289 - val_loss: 0.2256
Epoch 19/39
451/453 ————— 0s 25ms/step - accuracy: 0.8286 - loss: 0.3985
Epoch 19: val_loss did not improve from 0.21609
453/453 ————— 13s 28ms/step - accuracy: 0.8365 - loss: 0.3894 - val_accuracy: 0.9168 - val_loss: 0.2616
Epoch 20/39
452/453 ————— 0s 25ms/step - accuracy: 0.8355 - loss: 0.3931
Epoch 20: val_loss did not improve from 0.21609
453/453 ————— 13s 29ms/step - accuracy: 0.8373 - loss: 0.3876 - val_accuracy: 0.9228 - val_loss: 0.2368
Epoch 21/39
453/453 ————— 0s 25ms/step - accuracy: 0.8393 - loss: 0.3834
Epoch 21: val_loss did not improve from 0.21609
453/453 ————— 13s 28ms/step - accuracy: 0.8382 - loss: 0.3873 - val_accuracy: 0.9275 - val_loss: 0.2430
Epoch 22/39
452/453 ————— 0s 25ms/step - accuracy: 0.8361 - loss: 0.3950
Epoch 22: val_loss did not improve from 0.21609
453/453 ————— 13s 28ms/step - accuracy: 0.8371 - loss: 0.3914 - val_accuracy: 0.9279 - val_loss: 0.2258
Epoch 23/39
453/453 ————— 0s 25ms/step - accuracy: 0.8423 - loss: 0.3835
Epoch 23: val_loss did not improve from 0.21609
453/453 ————— 13s 29ms/step - accuracy: 0.8375 - loss: 0.3891 - val_accuracy: 0.9289 - val_loss: 0.2171
Epoch 24/39
451/453 ————— 0s 28ms/step - accuracy: 0.8420 - loss: 0.3722
Epoch 24: val_loss did not improve from 0.21609
453/453 ————— 14s 31ms/step - accuracy: 0.8366 - loss: 0.3852 - val_accuracy: 0.9200 - val_loss: 0.2287
Epoch 25/39
452/453 ————— 0s 25ms/step - accuracy: 0.8350 - loss: 0.3913
Epoch 25: val_loss improved from 0.21609 to 0.20922, saving model to best_Trans-Flat_model_LSTM_GOLD_\$10.keras
453/453 ————— 13s 29ms/step - accuracy: 0.8384 - loss: 0.3854 - val_accuracy: 0.9303 - val_loss: 0.2092
Epoch 26/39
451/453 ————— 0s 25ms/step - accuracy: 0.8358 - loss: 0.3886
Epoch 26: val_loss did not improve from 0.20922
453/453 ————— 13s 28ms/step - accuracy: 0.8374 - loss: 0.3839 - val_accuracy: 0.9224 - val_loss: 0.2422
Epoch 27/39
453/453 ————— 0s 25ms/step - accuracy: 0.8316 - loss: 0.3969
Epoch 27: val_loss did not improve from 0.20922
453/453 ————— 13s 28ms/step - accuracy: 0.8384 - loss: 0.3858 - val_accuracy: 0.9279 - val_loss: 0.2218
Epoch 28/39
453/453 ————— 0s 25ms/step - accuracy: 0.8418 - loss: 0.3811
Epoch 28: val_loss did not improve from 0.20922
453/453 ————— 13s 28ms/step - accuracy: 0.8372 - loss: 0.3860 - val_accuracy: 0.9233 - val_loss: 0.2220
Epoch 29/39
452/453 ————— 0s 27ms/step - accuracy: 0.8370 - loss: 0.3839
Epoch 29: val_loss did not improve from 0.20922
453/453 ————— 15s 33ms/step - accuracy: 0.8388 - loss: 0.3845 - val_accuracy: 0.9252 - val_loss: 0.2299

```

Epoch 29/39
452/453 — 0s 27ms/step - accuracy: 0.8370 - loss: 0.3839
Epoch 29: val_loss did not improve from 0.20922
453/453 — 15s 33ms/step - accuracy: 0.8388 - loss: 0.3845 - val_accuracy: 0.9252 - val_loss: 0.2299
Epoch 30/39
453/453 — 0s 28ms/step - accuracy: 0.8334 - loss: 0.3913
Epoch 30: val_loss did not improve from 0.20922
453/453 — 14s 31ms/step - accuracy: 0.8374 - loss: 0.3856 - val_accuracy: 0.9177 - val_loss: 0.2389
Epoch 31/39
452/453 — 0s 29ms/step - accuracy: 0.8365 - loss: 0.3887
Epoch 31: val_loss did not improve from 0.20922
453/453 — 15s 33ms/step - accuracy: 0.8359 - loss: 0.3886 - val_accuracy: 0.9307 - val_loss: 0.2257
Epoch 32/39
453/453 — 0s 30ms/step - accuracy: 0.8436 - loss: 0.3754
Epoch 32: val_loss did not improve from 0.20922
453/453 — 15s 34ms/step - accuracy: 0.8375 - loss: 0.3846 - val_accuracy: 0.9238 - val_loss: 0.2194
Epoch 33/39
453/453 — 0s 28ms/step - accuracy: 0.8336 - loss: 0.3838
Epoch 33: val_loss did not improve from 0.20922
453/453 — 14s 31ms/step - accuracy: 0.8377 - loss: 0.3840 - val_accuracy: 0.9289 - val_loss: 0.2516
Epoch 34/39
453/453 — 0s 26ms/step - accuracy: 0.8369 - loss: 0.3915
Epoch 34: val_loss did not improve from 0.20922
453/453 — 14s 30ms/step - accuracy: 0.8390 - loss: 0.3861 - val_accuracy: 0.9224 - val_loss: 0.2744
Epoch 35/39
452/453 — 0s 28ms/step - accuracy: 0.8434 - loss: 0.3779
Epoch 35: val_loss did not improve from 0.20922
453/453 — 14s 31ms/step - accuracy: 0.8376 - loss: 0.3873 - val_accuracy: 0.9252 - val_loss: 0.2695
Epoch 36/39
451/453 — 0s 27ms/step - accuracy: 0.8395 - loss: 0.3788
Epoch 36: val_loss did not improve from 0.20922
453/453 — 13s 30ms/step - accuracy: 0.8377 - loss: 0.3862 - val_accuracy: 0.9293 - val_loss: 0.2557
Epoch 37/39
452/453 — 0s 26ms/step - accuracy: 0.8416 - loss: 0.3808
Epoch 37: val_loss did not improve from 0.20922
453/453 — 14s 30ms/step - accuracy: 0.8375 - loss: 0.3874 - val_accuracy: 0.9252 - val_loss: 0.2434
Epoch 38/39
452/453 — 0s 29ms/step - accuracy: 0.8393 - loss: 0.3885
Epoch 38: val_loss did not improve from 0.20922
453/453 — 15s 32ms/step - accuracy: 0.8374 - loss: 0.3902 - val_accuracy: 0.9121 - val_loss: 0.3024
Epoch 39/39
452/453 — 0s 27ms/step - accuracy: 0.8375 - loss: 0.3961
Epoch 39: val_loss did not improve from 0.20922
453/453 — 20s 31ms/step - accuracy: 0.8350 - loss: 0.3956 - val_accuracy: 0.9233 - val_loss: 0.2407

```

```
LSTM_saved_best_model = keras.models.load_model('best_Trand-Flat_model_LSTM_GOLD_$10.keras')
```

```

pred = LSTM_saved_best_model.predict(X_test)
pred_binary = (pred >= 0.5).astype(int)

print(classification_report(y_test, pred_binary))

```

```

38/38 — 1s 18ms/step
              precision    recall  f1-score   support

     0.0         0.89        0.99         0.93        1058
     1.0         0.29        0.04         0.08         137

 accuracy          0.88        1195
 macro avg         0.59        0.51         0.51        1195
 weighted avg         0.82        0.88         0.84        1195

```

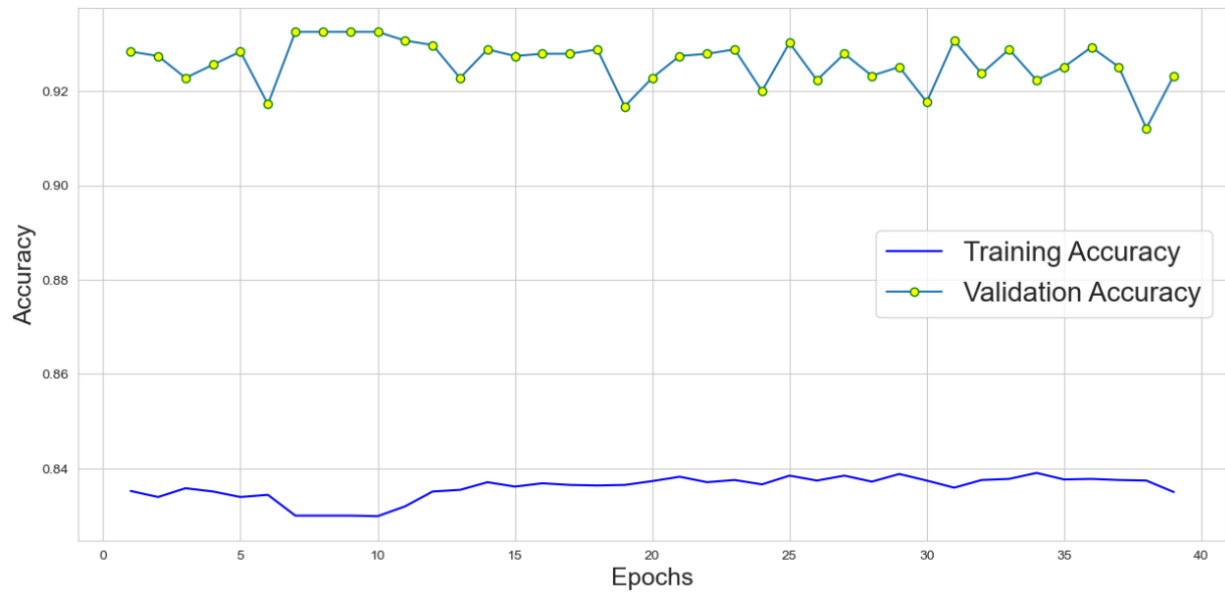
```

history_dict = history.history
Accuracy_values = history_dict['accuracy']
val_accuracy_values = history_dict['val_accuracy']

epochs = range(1, len(Accuracy_values) + 1)

plt.figure(figsize=(15,7))
plt.plot(epochs, Accuracy_values, 'b', label='Training Accuracy')
plt.plot(epochs, val_accuracy_values, marker='o', markeredgecolor='green', markerfacecolor='yellow', label='Validation Accuracy')
plt.xlabel('Epochs', size=18)
plt.ylabel('Accuracy', size=18)
plt.legend()
plt.show()

```



Week-11:


```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

X_train_lstm = X_train_imb.reshape((X_train_imb.shape[0], X_train_imb.shape[1], 1))
X_test_lstm = X_test_imb.reshape((X_test_imb.shape[0], X_test_imb.shape[1], 1))

model_lstm = Sequential()
model_lstm.add(LSTM(32, return_sequences=False, input_shape=(X_train_lstm.shape[1], 1)))
model_lstm.add(Dropout(0.2))
model_lstm.add(Dense(1, activation='sigmoid'))

model_lstm.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

history_lstm = model_lstm.fit(
    X_train_lstm, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

```

```

649/649 ————— 2s 1ms/step - accuracy: 0.7850 - loss: 0.5136 - val_accuracy: 0.8347 - val_loss: 0.4116
Epoch 2/10
649/649 ————— 1s 1ms/step - accuracy: 0.8328 - loss: 0.4042 - val_accuracy: 0.8401 - val_loss: 0.3911
Epoch 3/10
649/649 ————— 1s 1ms/step - accuracy: 0.8411 - loss: 0.3864 - val_accuracy: 0.8457 - val_loss: 0.3773
Epoch 4/10
649/649 ————— 1s 1ms/step - accuracy: 0.8407 - loss: 0.3874 - val_accuracy: 0.8473 - val_loss: 0.3721
Epoch 5/10
649/649 ————— 1s 1ms/step - accuracy: 0.8443 - loss: 0.3792 - val_accuracy: 0.8521 - val_loss: 0.3657
Epoch 6/10
649/649 ————— 1s 1ms/step - accuracy: 0.8512 - loss: 0.3728 - val_accuracy: 0.8498 - val_loss: 0.3680
Epoch 7/10
649/649 ————— 1s 1ms/step - accuracy: 0.8487 - loss: 0.3729 - val_accuracy: 0.8587 - val_loss: 0.3577
Epoch 8/10
649/649 ————— 1s 1ms/step - accuracy: 0.8550 - loss: 0.3648 - val_accuracy: 0.8614 - val_loss: 0.3523
Epoch 9/10
649/649 ————— 1s 1ms/step - accuracy: 0.8598 - loss: 0.3608 - val_accuracy: 0.8600 - val_loss: 0.3525
Epoch 10/10
649/649 ————— 1s 1ms/step - accuracy: 0.8598 - loss: 0.3546 - val_accuracy: 0.8642 - val_loss: 0.3477

```

```

: y_pred_lstm_prob = model_lstm.predict(X_test_lstm)
  y_pred_lstm = (y_pred_lstm_prob >= 0.5).astype(int)

  print("LSTM Classification Report:")
  print(classification_report(y_test, y_pred_lstm))

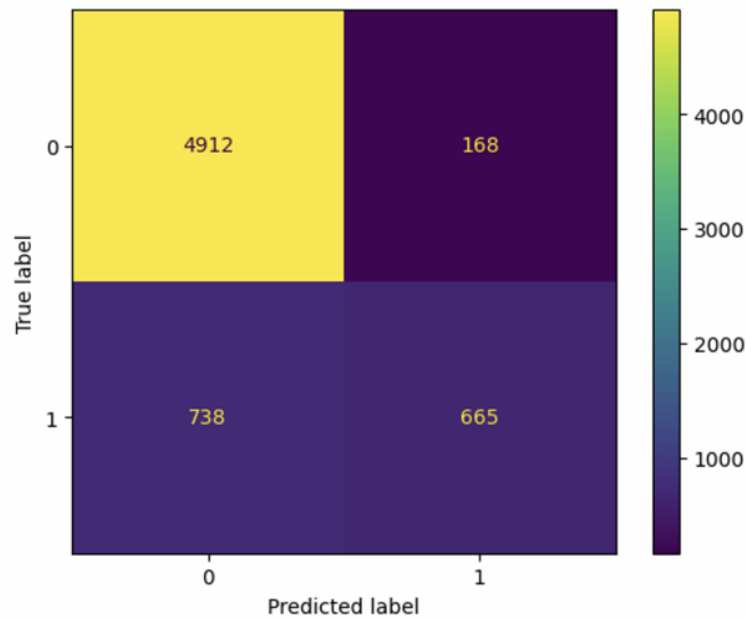
  cm_lstm = confusion_matrix(y_test, y_pred_lstm)
  disp_lstm = ConfusionMatrixDisplay(confusion_matrix=cm_lstm, display_labels=[0, 1])
  disp_lstm.plot()
  plt.show()

```

203/203 ————— 0s 873us/step

LSTM Classification Report:

	precision	recall	f1-score	support
0	0.87	0.97	0.92	5080
1	0.80	0.47	0.59	1403
accuracy			0.86	6483
macro avg	0.83	0.72	0.76	6483
weighted avg	0.85	0.86	0.85	6483



```
[96]: # Compute metrics for LSTM
acc_lstm = np.round(accuracy_score(y_test, y_pred_lstm) * 100, 2)
pre_lstm = np.round(precision_score(y_test, y_pred_lstm, average='weighted') * 100, 2)
rec_lstm = np.round(recall_score(y_test, y_pred_lstm, average='weighted') * 100, 2)
f1_lstm = np.round(f1_score(y_test, y_pred_lstm, average='weighted') * 100, 2)
auc_lstm = np.round(roc_auc_score(y_test, y_pred_lstm) * 100, 2)

# Add LSTM row
model_data_imb.loc[len(model_data_imb.index)] = [
    'LSTM', acc_lstm, pre_lstm, rec_lstm, f1_lstm, auc_lstm
]

model_data_imb
```

```
[96]:
```

	Imbalance AI Models	Accuracy	Precision	Recall	F1-Score	AUC
0	Random Forest	91.38	91.58	91.38	90.75	81.60
1	Logistic Regression	83.93	82.69	83.93	82.32	69.00
2	MLP	88.99	88.84	88.99	88.10	77.57
3	LSTM	86.02	85.40	86.02	84.61	72.05